

ComputeCredit v2.1

Corrected Architecture and Implementation Specification

Agent Revenue Factoring for the Agent Economy

Status: Corrected implementation specification

Author: Manus AI

Target: OKX Dev Day 2026 Hackathon — OKX AI Track with X Layer settlement

Build window: 17–25 September 2026

Primary deliverable: A working A+B MVP with an auditable revenue-servicing loop

One-line product definition: ComputeCredit is a short-duration, one-job compute-advance protocol in which a borrower agent receives the exact cost of a verified provider job and repays that advance from future marketplace revenue routed through an authorized split router.

Executive summary

ComputeCredit addresses a specific cash-flow problem in the agent economy. A newly created agent may have a paid service to deliver, but it may need to pay a provider agent for compute before it receives its first buyer payment. The agent therefore has an earning opportunity but cannot finance the first job.

The protocol solves this problem with a narrowly scoped advance. A lender deposits USDC into a vault. An authorized borrower or orchestrator requests an advance tied to one provider, one job, one quoted compute price, and one registered revenue source. The vault transfers the advance to the borrower's isolated spending wallet. When the borrower later receives an x402 payment, an authorized revenue router sends the configured share to the vault and forwards the remainder to the borrower.

The design is intentionally not a general-purpose lending protocol. It does not offer revolving credit, anonymous overcollateralized borrowing, or guaranteed lender returns. It offers a demonstration of **revenue-based servicing for a single compute obligation**.

The corrected version makes five important changes to the original design:

1. **Outstanding-principal accounting is defined once and cannot be double-decremented.**
2. **Lender ownership is represented with pool shares rather than raw deposits.**
3. **The orchestrator's authority is explicit and limited.**

4. Lien enforcement is described accurately as conditional on routed revenue.
 5. The MVP scope is reduced to the vault, a minimum trust passport, one router, one provider, and one end-to-end demo.
-

1. Product definition and scope

1.1 The problem

An agent called Nova sells a service for \$0.10 per request. To produce the service, Nova must call a provider endpoint that costs \$0.02. Nova has no starting balance. Without financing, Nova cannot pay the provider and therefore cannot complete the buyer's request.

The relevant problem is not a large unsecured loan. It is a small and time-bounded working-capital gap between:

- the provider payment required to perform a job; and
- the buyer payment expected after the job is delivered.

ComputeCredit advances only the provider cost for that job. It does not give the agent a general spending balance.

1.2 The product promise

ComputeCredit promises the following narrow behavior:

If an eligible agent has an active, priced job and insufficient funds, the protocol can advance the provider cost from a lender-funded pool and service that advance from future revenue routed through the protocol's authorized revenue source.

The protocol does **not** promise that every default will be recovered. A default creates a reputation penalty and a conditional future-revenue claim. If the agent never earns again through the registered route, the claim cannot recover funds that do not exist.

1.3 MVP boundary

Capability	MVP status	Implementation position
Lender deposits	Required	Onchain vault
Lender pool shares	Required	Simple proportional shares

One active advance per agent	Required	Onchain vault invariant
Provider price registry	Required	Prevents caller-supplied inflated quotes
Job binding	Required	Advance includes job hash and provider
Revenue servicing	Required	Authorized router calls the vault
Automatic settlement	Required	Exact principal accounting
Basic trust score	Required	Minimal passport with guarded seeding
Default and conditional lien	Required	Onchain record plus routed-revenue capture
Telegram interface	Recommended	Thin wrapper around orchestrator
Dashboard	Optional	Read-only event and state view
Work escrow	Stretch	Separate contract, not on critical path
Compute futures	Post-MVP	Not part of the hackathon core
Permissionless cross-wallet identity	Post-MVP	Not solved by this version
Fully automatic marketplace settlement	Post-MVP	Requires production router/facilitator integration

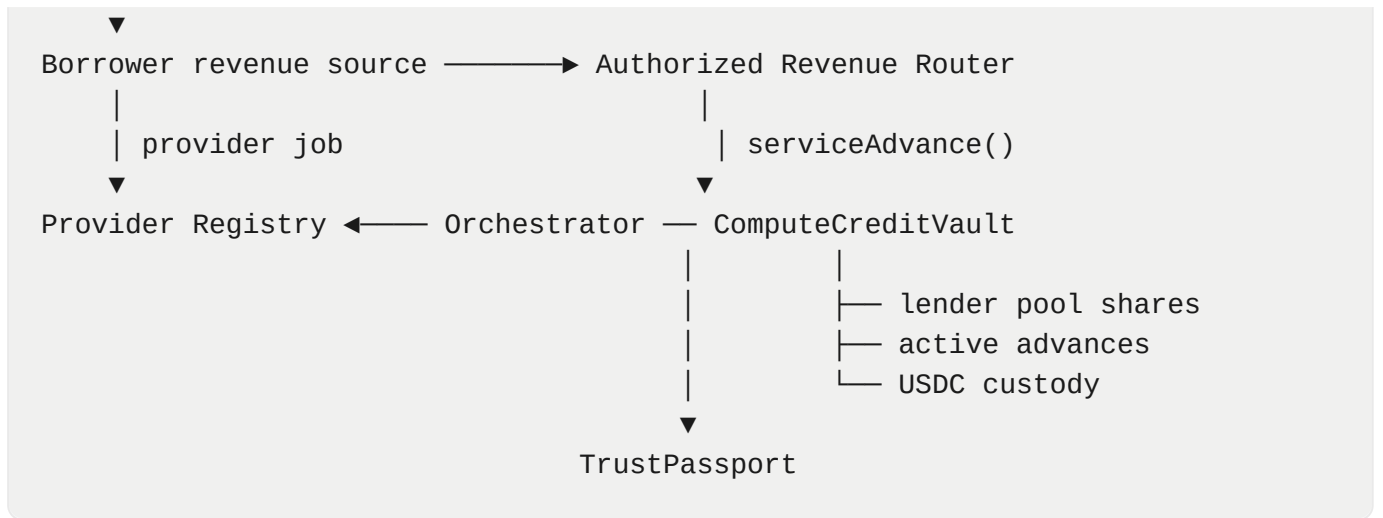
The submission should describe only the first seven capabilities as the core protocol. Stretch features must not be allowed to destabilize the core demo.

2. System architecture

2.1 Components

Plain Text

Buyer / agent
| pays through x402



Vault

The vault holds lender liquidity, issues one-job advances, records servicing, and tracks settlement or default. It is the source of truth for principal accounting.

TrustPassport

The passport stores a score and repayment-related attestations for a wallet identity. It is readable by other contracts, but it is not a complete identity or sybil-resistance system.

ProviderRegistry

The registry stores approved provider endpoints and their current maximum quoted price per job. The vault uses this registry to prevent an orchestrator from submitting an arbitrary or inflated provider quote.

RevenueRouter

The router receives or observes buyer revenue and sends the configured servicing amount to the vault. The router is the enforcement point for the demo. It must be explicitly authorized by the vault and cannot be treated as a universal lien-enforcement mechanism outside the registered route.

Orchestrator

The orchestrator coordinates the demo. It may be configured as an approved operator, but it cannot alter lender balances, change arbitrary scores, or issue an advance without satisfying the vault's onchain rules.

3. Roles and trust boundaries

Role	Responsibility	Trust level in MVP
Lender	Deposits USDC and owns pool shares	External user
Borrower agent	Requests or authorizes an advance and performs the job	External agent
Provider	Supplies compute at a registered price	Registered counterparty
Buyer	Pays the borrower for the delivered service	External payer
Vault	Custodies funds and enforces accounting	Onchain authority
Passport	Stores score and attestation data	Onchain registry
Revenue router	Splits routed revenue and services advances	Explicitly allowlisted
Orchestrator	Coordinates requests and demo UX	Trusted operator in MVP
Keeper	Calls permissionless default functions after expiry	External caller

3.1 Explicit MVP trust assumptions

The MVP has three deliberate centralization assumptions:

1. The orchestrator may submit a request for a borrower if the borrower has authorized that action or the demo wallet is controlled by the orchestrator.
2. The orchestrator may issue work attestations.
3. The revenue router controls the registered demo receiving path.

These assumptions must appear in the README and the demo narration. The protocol should be presented as a working prototype with a clear production migration path, not as a trustless production lending system.

3.2 Production migration path

A production version should replace the trusted assumptions with:

- agent-signed EIP-712 advance requests;
 - Agentic Wallet policy rules that constrain provider, amount, and destination;
 - a contract-controlled receiving wallet or facilitator-level split settlement;
 - multiple independent attesters for work history;
 - a verified provider quote or buyer escrow;
 - wallet and agent identity controls that reduce sybil resets.
-

4. Economic model

4.1 Pool accounting

The vault uses proportional pool shares. A lender's deposit is not stored as a permanently fixed withdrawal claim because pool assets can change as advances are issued, serviced, or recovered.

The vault tracks:

Plain Text

```
uint256 public totalShares;  
mapping(address => uint256) public shares;
```

The vault also tracks actual USDC held by the contract. For the MVP, outstanding advances are treated as receivables for reporting but are not assumed to be immediately liquid assets.

A conservative withdrawal rule is therefore:

Plain Text

```
withdrawableAssets = vault USDC balance
```

A lender may withdraw only the proportional share of currently idle USDC. Outstanding advances remain obligations of borrowers and do not become immediately withdrawable liquidity.

4.2 Share issuance

Let A be the vault's current idle USDC balance and S be total shares before a deposit.

- If $S == 0$, mint shares equal to the deposit amount.

- Otherwise, mint $\text{depositAmount} \times S / A$ shares.
- If $A == 0$ while $S > 0$, revert because the pool is insolvent or the accounting is invalid.

For a withdrawal of `amount`, burn:

Plain Text

```
sharesToBurn = amount × totalShares / idleAssets
```

The implementation must round in the direction that protects remaining shareholders. The selected rounding rule must be documented and tested.

4.3 Advance accounting invariant

For every active advance:

Plain Text

```
0 ≤ serviced ≤ principal
```

The vault's `totalOutstanding` means **the sum of unserviced principal across active, non-defaulted advances**.

When an advance is created:

Plain Text

```
totalOutstanding += principal
```

When an amount is serviced:

Plain Text

```
totalOutstanding -= amount
```

When an advance defaults, only the remaining unserviced principal is removed from `totalOutstanding`:

Plain Text

```
shortfall = principal - serviced
totalOutstanding -= shortfall
```

Settlement does not perform a second principal subtraction. This rule eliminates the double-decrement error in the previous specification.

4.4 Recovery accounting

A repayment transfers USDC to the vault and increases idle liquidity. It does not automatically increase a lender's individual balance. Instead, all shareholders participate proportionally through their pool shares.

A default does not guarantee recovery. It creates a lien target:

Plain Text

```
lienTarget = shortfall × 1.5
```

The target is a recovery objective, not a guaranteed asset. Any future revenue captured through the authorized router is deposited into the vault and credited toward the lien target.

The document should never state that lenders are made whole in every path. The accurate statement is:

Defaults create an enforceable recovery attempt against future revenue routed through the registered source, but recovery depends on the agent generating future revenue.

5. Contract specifications

5.1 ComputeCreditVault state

The following is the minimum state model. Production code should use OpenZeppelin SafeERC20 , access control, and reentrancy protection.

Plain Text

```
interface IERC20 {
    function transfer(address to, uint256 amount) external returns (bool);
    function transferFrom(address from, address to, uint256 amount) external re
}

struct ComputeAdvance {
    uint256 principal;
    uint256 serviced;
    uint256 issuedAt;
```

```

uint256 dueAt;
uint256 splitBps;
bytes32 jobHash;
address borrower;
address provider;
address revenueSource;
bool settled;
bool defaulted;
}

IERC20 public immutable usdc;
TrustPassport public immutable passport;
ProviderRegistry public immutable providers;

uint256 public constant MIN_DEPOSIT = 1e6;
uint256 public constant MAX_BPS = 10_000;
uint256 public constant DEFAULT_MULTIPLIER_BPS = 15_000;
uint256 public constant NORMAL_SPLIT_BPS = 2_000;
uint256 public constant ADVANCE_WINDOW = 43_200;
uint256 public constant SCORE_GRANULARITY = 1_000;

uint256 public totalShares;
uint256 public totalOutstanding;

mapping(address => uint256) public shares;
mapping(address => ComputeAdvance) public advances;
mapping(address => bool) public approvedRouters;
mapping(address => bool) public approvedOperators;

```

5.2 Vault invariants

The implementation and tests must enforce these invariants:

Invariant	Meaning
<code>served <= principal</code>	No advance can be over-served
One active advance per borrower	Prevents stacking risk
<code>totalOutstanding</code> equals unserved principal sum	Utilization is reliable
Only approved router services an advance	Prevents arbitrary repayment records
Only approved operator or signed borrower request can issue	Prevents unauthorized borrowing

Settled/defaulted advances cannot be serviced	Prevents post-closure state changes
Lien capture cannot exceed target	Prevents excess seizure
Only vault can change score after deployment	Keeps score authority bounded
Only idle USDC can be withdrawn	Prevents withdrawals from draining live advances

5.3 Deposit liquidity

Plain Text

```
depositLiquidity(uint256 amount)
```

Requirements:

- `amount >= MIN_DEPOSIT` ;
- `transferFrom` succeeds;
- share issuance does not divide by zero except for the initial deposit;
- the contract emits `LiquidityDeposited` with the amount and shares minted.

The function must use a non-reentrant guard.

5.4 Withdraw liquidity

Plain Text

```
withdrawLiquidity(uint256 amount)
```

Requirements:

- the caller has sufficient shares;
- `amount <= idleAssets × callerShares / totalShares` ;
- the amount is transferred using `SafeERC20` ;
- the correct shares are burned;
- the contract emits `LiquidityWithdrawn` .

The function must not use `totalDeposits - totalOutstanding` as a substitute for actual idle assets. That expression does not correctly model pool-share ownership or repayments.

5.5 Request an advance

The recommended MVP interface is operator-assisted but borrower-specific:

Plain Text

```
requestComputeAdvanceFor(  
    address borrower,  
    address provider,  
    uint256 computeCost,  
    bytes32 jobHash,  
    address revenueSource  
)
```

The function must perform the following checks:

1. The caller is an approved operator or the borrower has authorized the request.
2. The borrower has no active unsettled and non-defaulted advance.
3. The provider is active in the provider registry.
4. `computeCost` is no greater than the provider's registered price.
5. `computeCost` is no greater than the borrower's score tier limit.
6. `revenueSource` is registered for the borrower or is an approved router-controlled source.
7. The pool has sufficient idle liquidity.
8. `computeCost` is non-zero.
9. `jobHash` has not already been used for an advance.

The function then:

- stores the full `ComputeAdvance` record;
- increments `totalOutstanding` by exactly `computeCost` ;
- transfers USDC to the borrower's isolated spending wallet or approved destination;
- emits `AdvanceRequested` and `RevenueSourceRegistered` .

The advance must be bound to a provider and job hash. A free-form `providerQuote` calldata parameter is not sufficient because a caller could supply an inflated quote.

5.6 Service an advance

Plain Text

```
serviceAdvance(address borrower, uint256 amount)
```

Only an approved revenue router may call this function.

Requirements:

- the borrower has an active advance;
- the advance is not settled or defaulted;
- `amount > 0` ;
- `amount <= principal - serviced` ;
- the router transfers the USDC to the vault before or as part of the call;
- `totalOutstanding` is reduced by exactly `amount` ;
- `serviced` is increased by exactly `amount` .

If `serviced == principal` , the function:

- sets `settled = true` ;
- sets the borrower' s active position to closed;
- increases the score by the configured on-time amount, capped at 1,000;
- emits `AdvanceSettled` .

There must be no additional `totalOutstanding -= principal` statement during settlement.

A safer production interface is:

Plain Text

```
serviceAdvanceWithTransfer(address borrower, uint256 amount)
```

which performs `transferFrom(router, address(this), amount)` and then updates state. This reduces the risk of recording repayment without receiving tokens.

5.7 Early repayment

Plain Text

```
repayEarly(address borrower, uint256 amount)
```

For the MVP, the borrower may repay the remaining principal directly if:

- the caller is the borrower or an approved operator;

- the transfer succeeds;
- the amount is no greater than the remaining principal.

The same servicing and settlement path should be reused internally so that score updates and outstanding accounting cannot diverge between early repayment and revenue repayment.

5.8 Default

Plain Text

```
penalize(address borrower)
```

This function is permissionless after expiry.

Requirements:

- an active advance exists;
- `block.timestamp > dueAt` or the selected time primitive has passed;
- the advance is not settled or already defaulted;
- remaining principal is greater than zero.

Effects:

Plain Text

```
shortfall = principal - serviced
principalAfterDefault = 0 for outstanding-accounting purposes
totalOutstanding -= shortfall
advance.defaulted = true
lienTarget = shortfall × 15,000 / 10,000
lienCaptured = 0
revenueLienBps = 10,000
passport score is slashed according to the configured rule
```

The default event must include the shortfall and lien target. The default function must not transfer additional money to the borrower or lender. Recovery occurs only when future revenue is captured.

5.9 Lien capture

Plain Text

```
captureLien(address borrower, uint256 amount)
```

Only an approved router may call this function.

The router transfers the payment to the vault. The vault calculates:

Plain Text

```
remainingTarget = lienTarget - lienCaptured
captured = min(amount, remainingTarget)
remainder = amount - captured
```

The router forwards `remainder` to the borrower's destination. When `lienCaptured == lienTarget`, the vault clears the lien:

Plain Text

```
revenueLienBps[borrower] = 0;
```

If the defaulted borrower also has a new advance, the MVP should reject that request until the lien is cleared or a governance-defined rehabilitation process is completed.

6. Provider registry

The provider registry is required for quote integrity.

Plain Text

```
struct ProviderQuote {
    bool active;
    address providerWallet;
    uint256 pricePerJob;
    uint256 updatedAt;
    bytes32 serviceId;
}

mapping(address => ProviderQuote) public providerQuotes;
```

The registry owner or approved administrator may register a provider and update its price. Every update emits an event.

An advance request must reference the provider address. The vault checks the provider's registered price at request time. The requested compute cost may be lower than the

registered price but may not exceed it.

For stronger job binding, the orchestrator should compute:

Plain Text

```
jobHash = keccak256(  
    borrower,  
    provider,  
    serviceId,  
    quotedPrice,  
    nonce,  
    expiry  
)
```

The provider or borrower should sign the job authorization offchain. The MVP may use the orchestrator as the trusted job coordinator, but the job hash must still be stored onchain.

7. TrustPassport specification

7.1 What the passport is

The passport is a wallet-linked reputation registry. It stores a score from 0 to 1,000 and hashes of work or repayment attestations.

It does not prove that two wallets belong to the same human or agent. It does not eliminate sybil attacks. It makes the score and recorded history readable by other contracts.

7.2 State

Plain Text

```
mapping(address => uint256) public score;  
mapping(address => bool) public scoreSeeded;  
mapping(address => bytes32[]) public attestations;  
mapping(address => uint256) public revenueLienBps;  
mapping(address => uint256) public lienTarget;  
mapping(address => uint256) public lienCaptured;  
  
address public vault;  
address public attester;
```

7.3 Score rules

The score is divided into four underwriting tiers:

Score	Maximum advance	Interpretation
0–200	1 USDC	New or minimally established wallet
201–500	5 USDC	Some successful history
501–800	25 USDC	Established wallet
801–1,000	50 USDC	Strong recorded history

The amount must also be limited by the one-job provider quote. The tier limit is never a general credit line.

7.4 Seeding

Plain Text

```
seedScore(address agent, uint256 initialScore)
```

Requirements:

- callable only by the configured attester;
- callable only once per wallet;
- `initialScore` is clamped to 0–200;
- `scoreSeeded[agent]` is set before emitting the event;
- the event reason identifies the score as a bootstrap value.

The fallback score of 150 is acceptable for a hackathon demonstration when external wallet-history data is unavailable. It must be labeled as a trusted bootstrap value.

7.5 Score changes

On-time settlement increases the score by a fixed amount, for example 20, capped at 1,000.

Default handling should be deterministic. For the demo, a full slash to zero is simple to explain. A production system should use a more carefully calibrated penalty and possibly distinguish fraud, expiry, and temporary illiquidity.

Only the vault may call score mutation functions after deployment.

7.6 Attestations

A minimum attestation structure is:

Plain Text

```
struct WorkAttestation {  
    address agent;  
    bytes32 workHash;  
    bytes32 paymentTxHash;  
    uint256 issuedAt;  
    uint256 nonce;  
}
```

The EIP-712 domain must include:

- name;
- version;
- chain ID;
- verifying contract.

The typed data must include the agent, work hash, payment transaction hash, timestamp, and nonce. The nonce prevents replay. The contract or an offchain verifier must confirm that the signer is the configured attester.

For the MVP, it is acceptable to store the attestation hash onchain and verify the signature offchain. The README must not claim full onchain signature verification unless that function is implemented and tested.

8. Revenue router

8.1 Normal servicing

For a normal active advance:

Plain Text

```
servicingAmount = buyerPayment × splitBps / 10,000  
borrowerAmount = buyerPayment - servicingAmount
```

The router calls the vault's servicing function and forwards the remainder to the borrower.

The router must verify:

- the borrower's registered revenue source;
- the current advance state;
- whether a lien is active;
- the configured split basis points;
- the payment amount and token.

8.2 Defaulted servicing

For a borrower with an active lien, the MVP router sends 100% of routed revenue toward the outstanding lien target. Any amount above the remaining target is forwarded to the borrower.

This behavior applies only to payments that pass through the registered revenue source. It does not automatically capture payments sent to unrelated wallets or marketplaces.

8.3 Router authorization

The vault must expose an explicit administrator-controlled allowlist:

Plain Text

```
setApprovedRouter(address router, bool approved)
```

The function should emit an event. Router replacement must be visible in the event history and documented in the deployment configuration.

The router must not be able to:

- alter the borrower's score;
- create an advance;
- change a provider quote;
- withdraw lender funds;
- change another borrower's revenue source.

9. End-to-end flows

9.1 Happy-path factoring flow

1. A lender deposits 50 USDC.
2. The vault mints proportional pool shares.
3. Nova has a bootstrap score of 150 and a tier limit of 1 USDC.
4. The provider registry reports a price of 0.02 USDC for the selected job.
5. Nova or the approved orchestrator submits a request for a 0.02 USDC advance.
6. The vault verifies the provider, job hash, score tier, revenue source, and idle liquidity.
7. The vault transfers 0.02 USDC to Nova's isolated spending destination.
8. The orchestrator pays the provider through the configured payment rail.
9. Nova completes the buyer's service.
10. The buyer pays Nova 0.10 USDC.
11. The router sends 20% of the payment, or 0.02 USDC, to the vault.
12. The vault increases `served` to 0.02 and decreases `totalOutstanding` to zero.
13. The advance is marked settled.
14. Nova's score increases by 20, capped at 1,000.
15. The router forwards the remaining 0.08 USDC to Nova.

9.2 Partial servicing flow

If the buyer payment is 0.05 USDC, the normal 20% split services only 0.01 USDC.

The vault records:

Plain Text

```
principal = 0.02
served = 0.01
remaining = 0.01
totalOutstanding decreases by 0.01
settled = false
```

A later routed payment may service the remaining 0.01 USDC. The score is not increased until the principal is fully serviced.

9.3 Default flow

1. Nova receives an advance of 0.02 USDC.

2. Nova produces no routed revenue before `dueAt` .
3. Anyone calls `penalize(Nova)` after expiry.
4. The vault records a shortfall of 0.02 USDC.
5. `totalOutstanding` decreases by 0.02 because the active advance leaves the outstanding-principal set.
6. Nova's score is slashed.
7. The lien target becomes 0.03 USDC.
8. The registered router sets the revenue capture rate to 100%.
9. Nova later receives 0.01 USDC through the registered route.
10. The vault records 0.01 USDC of lien recovery.
11. Nova receives no remainder because the lien still has 0.02 USDC remaining.
12. After 0.03 USDC has been captured, the lien clears and later revenue is no longer captured by the lien.

9.4 Lender flow

1. The lender deposits USDC and receives pool shares.
 2. The lender monitors idle assets, total shares, and outstanding principal.
 3. Repayments increase vault liquidity and therefore increase the value available to shareholders.
 4. The lender withdraws only the amount permitted by the lender's share of idle assets.
 5. The lender does not receive a guaranteed return. Any penalty recovery is reflected in pool assets and distributed proportionally through the share model.
-

10. Security model

10.1 Required protections

The Solidity implementation must include:

- `SafeERC20` for USDC transfers;
- `ReentrancyGuard` for token-moving functions;
- role-based access control for the vault administrator, operator, router, registry, and

passport;

- zero-address validation;
- explicit token-address validation;
- checks-effects-interactions ordering;
- protection against duplicate settlement;
- protection against repayment after default;
- protection against changing the revenue source while an advance is active;
- protection against reusing a job hash;
- emergency pause support for critical token-moving functions;
- events for all state transitions.

10.2 Threat model

Threat	Impact	MVP mitigation	Production improvement
Inflated provider quote	Excess advance	Onchain provider registry	Signed provider quote and buyer escrow
Unauthorized advance	Pool loss	Approved operator or borrower authorization	EIP-712 agent signatures and wallet policy
Fake repayment record	False settlement	Router must transfer USDC with servicing	Atomic facilitator settlement
Router compromise	Misrouted revenue	Small allowlisted demo router	Audited contract and policy-controlled wallet
Sybil wallet reset	Repeated low-tier borrowing	One advance, low cap, slow score growth	Agent identity and stake/history proofs
Default with no future revenue	Lender loss	Conditional lien and reputation slash	Escrowed receivable or collateral
Reentrancy	Accounting or token loss	Reentrancy guard and SafeERC20	Audit and invariant testing
Incorrect decimals	Mispriced amounts	One token-decimal configuration	Deployment-time assertions

Double settlement	Corrupt accounting	Central settlement function and tests	Formal verification of invariants
-------------------	--------------------	---------------------------------------	-----------------------------------

10.3 Core test invariants

Foundry tests should assert:

Plain Text

```
served <= principal
settled implies served == principal
defaulted implies settled == false
totalOutstanding equals the sum of remaining principal for active advances
lienCaptured <= lienTarget
cleared lien implies revenueLienBps == 0
non-router service attempts revert
second active advance attempts revert
withdrawals cannot exceed idle liquidity
```

11. Offchain implementation

11.1 Orchestrator lifecycle

The orchestrator performs the following sequence:

1. Receive an inference request.
2. Identify the borrower, provider, service, and job hash.
3. Read the borrower balance.
4. Read the borrower score and maximum advance.
5. Read the provider's registered price.
6. Pay directly if the borrower has sufficient funds.
7. Otherwise request one compute advance through the approved path.
8. Pay the provider.
9. Return the result to the buyer.
10. Watch the registered revenue source.

11. Route the servicing amount.
12. Record the payment and work attestation.
13. Monitor expiry and submit `penalize` when appropriate.

The orchestrator should never calculate a permission that the vault does not enforce. Offchain checks are for user experience; onchain checks are the authority.

11.2 Provider registry configuration

For the demo, use one provider endpoint:

JSON

```
{
  "providerAddress": "0x...",
  "serviceId": "haiku-v1",
  "pricePerJobUsdc": "0.02",
  "endpoint": "https://provider.example/infer",
  "active": true
}
```

The actual endpoint and addresses must be stored in environment variables or deployment configuration, not hardcoded in source code.

11.3 Telegram commands

Command	Function
<code>/infer <prompt></code>	Runs the complete quote, advance, provider, and response flow
<code>/invest <amount></code>	Deposits USDC and displays shares received
<code>/position</code>	Displays shares, estimated idle claim, and pool status
<code>/withdraw <amount></code>	Withdraws available idle liquidity
<code>/score</code>	Displays score, tier, active advance, and lien status
<code>/history</code>	Displays advances, servicing events, settlements, defaults, and attestations

/pool

Displays idle assets, total shares, outstanding principal, and utilization

The bot is a user interface. It must not bypass contract checks or sign arbitrary transactions without policy controls.

12. Dashboard requirements

A dashboard is optional and should be built only after the core loop works.

The dashboard should display:

- idle USDC in the vault;
- total pool shares;
- total outstanding principal;
- active advances;
- serviced and remaining amounts;
- borrower scores;
- defaulted borrowers and lien targets;
- event links to the relevant explorer;
- router and provider configuration;
- explicit “demo trust assumptions” notice.

The dashboard must distinguish between:

- **actual vault liquidity;**
- **outstanding receivables;**
- **conditional lien targets.**

These values must not be combined into a single “guaranteed assets” number.

13. Stretch module: WorkEscrow

WorkEscrow should remain separate from the core vault. It can provide a stronger underwriting narrative because the buyer’s payment is locked before the worker requests compute.

The minimum escrow flow is:

1. The client deposits USDC.
2. The escrow stores the job hash, worker, amount, deadline, and arbiter.
3. The worker delivers a result hash.
4. The client confirms or disputes.
5. The escrow releases funds to the worker or resolves according to the arbiter rule.
6. The release can pass through the revenue router to service a compute advance.

The escrow module must not be included in the core “working demo” claim unless it is deployed and tested. If it is not shipped, describe it as a post-MVP design.

14. Deployment plan

Day 1: integration gates

- Confirm the current X Layer testnet chain ID, RPC, explorer, token address, and faucet.
- Confirm the current payment SDK and facilitator flow.
- Complete one provider payment round trip.
- Create the repository and environment configuration.

Gate: if a payment round trip does not work, do not begin optional modules.

Day 2: contract foundations

- Implement `TrustPassport` .
- Implement `ProviderRegistry` .
- Implement vault deposits, shares, and withdrawals.
- Add access control and reentrancy protection.

Day 3: advance lifecycle

- Implement request validation.
- Bind advances to provider and job hash.
- Implement exact `totalOutstanding` accounting.
- Add happy-path and rejection tests.

Day 4: router and servicing

- Implement the authorized revenue router.
- Implement partial servicing and exact settlement.
- Add score bump on settlement.
- Test atomic token transfer and accounting.

Day 5: default and lien

- Implement expiry and permissionless penalization.
- Implement conditional lien targets.
- Implement capture and automatic clearing.
- Test partial recovery and over-target payments.

Day 6: orchestrator and bot

- Implement provider quote, balance check, and advance decision.
- Implement one provider endpoint.
- Add `/infer` , `/invest` , `/score` , `/position` , and `/history` .

Day 7: testnet integration

- Deploy the contracts.
- Run the full happy path on testnet.
- Run partial servicing.
- Run default and recovery.
- Record transaction hashes.

Day 8: hardening and presentation

- Complete invariant tests.
- Rehearse the demo three times.
- Prepare a fallback recording.
- Build the dashboard only if all core tests are green.

Day 9: submission

- Freeze the contract configuration.
 - Publish the README with deployed addresses and trust assumptions.
 - Record the final demo.
 - Submit only features that are demonstrably working.
-

15. Testing checklist

Contract tests

- Deposit below minimum reverts.
- Initial deposit mints correct shares.
- Subsequent deposit mints proportional shares.
- Withdrawal cannot exceed idle share value.
- Non-approved operator cannot request an advance.
- Borrower cannot have two active advances.
- Unknown provider reverts.
- Advance above registered provider price reverts.
- Advance above score tier reverts.
- Reused job hash reverts.
- Pool with insufficient idle liquidity reverts.
- Non-router servicing reverts.
- Partial servicing updates `serviced` and `totalOutstanding` exactly once.
- Exact final servicing settles the advance.
- Settlement increases score once.
- Post-settlement servicing reverts.
- Early repayment uses the same accounting path.
- Penalization before expiry reverts.
- Penalization after expiry records the correct shortfall.
- Repeated penalization reverts.

- Lien target equals 1.5 times shortfall.
- Lien capture cannot exceed target.
- Lien clears at the target.
- Excess payment after lien clearance reaches the borrower.
- Score seeding can occur only once.
- Score cannot exceed 1,000.
- Score cannot become negative.
- Token transfers use the configured USDC token.
- Reentrancy attempts revert.

Integration tests

- Provider payment succeeds on the selected testnet.
 - Advance request reaches the vault.
 - Provider receives the exact registered cost.
 - Buyer payment reaches the router.
 - Router services the correct borrower.
 - Borrower receives the correct remainder.
 - Explorer events match the dashboard.
 - Telegram output matches onchain state.
 - Default recovery is visible in the explorer and dashboard.
-

16. Demo script

0:00–0:20 — Problem

“Agents can earn per job, but they may need to pay for compute before receiving their first buyer payment. ComputeCredit advances one verified compute cost and services it from future routed revenue.”

0:20–0:50 — Lender

Use `/invest 50` . Show the vault' s idle liquidity and the lender' s pool shares. Explain that the lender owns a proportional claim on pool assets and does not receive a guaranteed return.

0:50–1:50 — Happy path

Show Nova with a zero balance. Display the provider' s registered price of 0.02 USDC. Request the advance. Show the provider payment and the returned service result.

1:50–2:40 — Revenue servicing

Send a simulated buyer payment of 0.10 USDC through the registered route. Show 0.02 USDC servicing the advance and 0.08 USDC reaching Nova. Show `serviced == principal` , settlement, and the score increase.

2:40–3:20 — Partial servicing or default

If possible, demonstrate a smaller buyer payment and show partial servicing. Otherwise, show an expired advance, score slash, lien target, and a later routed payment being captured.

3:20–4:00 — Honest architecture close

Explain that the MVP uses an approved orchestrator and revenue router. State that production would replace these assumptions with agent signatures, wallet policies, escrowed receivables, and automatic facilitator settlement.

17. Judge questions and accurate answers

Who funds the advances?

Lenders deposit USDC into a pooled vault and receive proportional pool shares. Advances are limited by idle liquidity and borrower eligibility.

How does an agent repay if it is broke?

The agent does not need to repay from its existing balance. The registered revenue router services the advance from future buyer payments. The repayment mechanism is conditional on revenue being received through that route.

Is this a faucet?

No. The design has a lender-funded pool, a defined principal obligation, a servicing rule, and a default recovery mechanism. It is not risk-free, and the protocol does not claim that every default is recovered.

Can the agent avoid the lien by changing wallets?

In the MVP, yes, payments outside the registered route are not automatically captured. This is a known limitation. Production enforcement requires a contract-controlled receiving wallet, wallet policy, or facilitator-level split settlement.

Can the orchestrator steal funds?

The MVP trusts the orchestrator for coordination and attestation, but the vault limits what the orchestrator can do. It cannot change lender shares, bypass provider-price checks, or record repayment without the router authorization. Production should reduce this trust with agent signatures and policy-controlled wallets.

Is the trust score sybil resistant?

Only partially. The score is portable for a wallet identity, but a new wallet can begin again at the bootstrap tier. Strong cross-wallet identity and sybil resistance are future work.

Are lenders guaranteed to be made whole?

No. The protocol attempts recovery through future routed revenue and applies a reputation penalty, but recovery depends on the borrower earning again. Escrowed receivables would provide stronger protection.

Why is this factoring?

The protocol advances cash against an expected future receivable and collects from that receivable when it arrives. The analogy is limited: the MVP is an onchain revenue-based advance, not a regulated financial factoring product.

18. Repository structure

Plain Text

```
computecredit/  
├─ README.md  
├─ contracts/
```

```

├── src/
│   ├── ComputeCreditVault.sol
│   ├── TrustPassport.sol
│   ├── ProviderRegistry.sol
│   ├── RevenueRouter.sol
│   └── WorkEscrow.sol          # stretch only
├── test/
│   ├── ComputeCreditVault.t.sol
│   ├── TrustPassport.t.sol
│   ├── RevenueRouter.t.sol
│   └── Invariants.t.sol
├── script/Deploy.s.sol
├── foundry.toml
├── orchestrator/
│   ├── index.ts
│   ├── providerRegistry.json
│   ├── revenueRouter.ts
│   └── config.ts
├── bot/
│   └── telegram.ts
├── dashboard/
├── docs/
│   ├── ARCHITECTURE.md
│   ├── THREAT_MODEL.md
│   └── DEMO.md
└── .env.example

```

The README must include:

- the one-paragraph product explanation;
- the architecture diagram;
- deployed contract addresses;
- network and token configuration;
- exact demo steps;
- test commands;
- the trust assumptions;
- the known limitations;
- a clear separation between shipped and planned modules.

19. Final release criteria

The MVP is ready for submission only when all of the following are true:

1. A lender can deposit and receive pool shares.
 2. A lender cannot withdraw liquidity backing an active advance.
 3. A valid borrower can request one advance tied to a registered provider and job hash.
 4. An invalid quote, provider, score tier, or duplicate job is rejected.
 5. A router can service an advance only after transferring USDC.
 6. Partial servicing updates principal accounting exactly once.
 7. Exact servicing settles the advance and bumps the score once.
 8. Default records a correct shortfall and conditional lien.
 9. Lien capture stops at the exact target.
 10. The happy path works on the selected testnet.
 11. The default path works on the selected testnet.
 12. The README accurately describes all trust assumptions.
 13. The demo never claims guaranteed recovery or universal lien enforcement.
-

20. Final positioning

ComputeCredit should be positioned as:

A prototype credit primitive for the agent economy: a lender-funded, one-job compute advance serviced from routed agent revenue.

The strongest submission is not the one with the most modules. It is the one that makes the following sequence undeniable and auditable:

Plain Text

```
lender deposits → provider price is verified → one compute advance is issued  
→ provider is paid → buyer revenue arrives → router services the advance  
→ vault settles exactly → score changes exactly
```

The escrow and futures modules remain valuable future extensions, but they should not distract from the core proof. A reliable vault, a correct router, and honest trust boundaries are more persuasive than a larger but partially implemented protocol.

References

- [1] Solidity Documentation
- [2] OpenZeppelin Contracts Documentation
- [3] EIP-712: Typed Structured Data Hashing and Signing
- [4] Foundry Book
- [5] X Layer Documentation

External product names, network parameters, token addresses, SDK behavior, and payment-facilitator behavior must be confirmed against current official documentation immediately before deployment.